BRACT's

Vishwakarma Institute of Information Technology, Pune Practical Implementation Sheet

Department: CS-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 2 Practical No:3 Roll no: 21 PRN No: 12411682

Subject Name: Deep Learning

Name of Student: Pranay Zunzunwala

Problem Statement: Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance.

Algorithm:

Step 1: Import the required libraries such as NumPy, Matplotlib, TensorFlow/Keras, and the necessary modules from scikit-learn.

Step 2: Load the Diabetes dataset from the scikit-learn library and separate it into input features (X) and target values (y).

Step 3: Split the dataset into training and testing sets using an **80:20 ratio**.

Step 4: Standardize the training and testing feature data using **StandardScaler** to improve the convergence and performance of the neural network.

Step 5: Create a **Sequential Neural Network** using TensorFlow/Keras by defining an input layer, multiple hidden layers with **ReLU** activation, and an output layer with a **Linear** activation function for regression.

Step 6: Compile the model using the **Adam optimizer** with a specified learning rate and **Mean Squared Error (MSE)** as the loss function, then train the model to perform forward propagation and backpropagation.

Step 7: Train the model using different **learning rates** while keeping the number of epochs fixed, and record the training and validation loss along with the evaluation metrics.

Step 8: Train the model using different **numbers of epochs** while keeping the learning rate fixed, and observe the effect on model performance.

Step 9: Use the trained model to predict the target values for the testing dataset and evaluate the model using **Mean Squared Error (MSE), Mean Absolute Error (MAE), and R^2 Score**.

Step 10: Visualize the **training and validation loss curves** and plot the **Actual vs. Predicted** values, then analyze the effect of different learning rates and epochs to determine the best-performing model configuration.

Code :

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
diabetes = load_diabetes()

X = diabetes.data
y = diabetes.target

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size = 0.2, random_state = 42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

def create_model(learning_rate):
    model = Sequential([
        Dense(64, activation='relu', input_shape=(X_train.shape[1],)), Dense(32, activation='relu'),
Dense(32, activation='relu'), Dense(16, activation='relu'), Dense(1, activation='linear')
    ])

    optimizer = Adam(learning_rate=learning_rate)

    model.compile(
```

```
        optimizer=optimizer,
        loss='mse',
        metrics=['mae']
    )

    return model

learning_rates = [0.1, 0.01, 0.001, 0.0001]

print("EFFECT OF LEARNING RATE")

for lr in learning_rates:

    print(f"\nTraining with Learning Rate = {lr}")

    model = create_model(lr)

    history = model.fit(
        X_train,
        y_train,
        validation_split=0.2,
        epochs=50,
        batch_size=16,
        verbose=0
    )

    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

    print("MSE :", mse)
    print("MAE :", mae)
    print("R2  :", r2)

    plt.figure(figsize=(6,4))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f'Learning Rate = {lr}')
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
```

```
    plt.show()
epochs_list = [10, 30, 50, 100]

print("EFFECT OF EPOCHS\n")

for epochs in epochs_list:

    print(f"\nTraining with Epochs = {epochs}")

    model = create_model(0.001)

    history = model.fit(
        X_train,
        y_train,
        validation_split=0.2,
        epochs=epochs,
        batch_size=16,
        verbose=0
    )

    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)

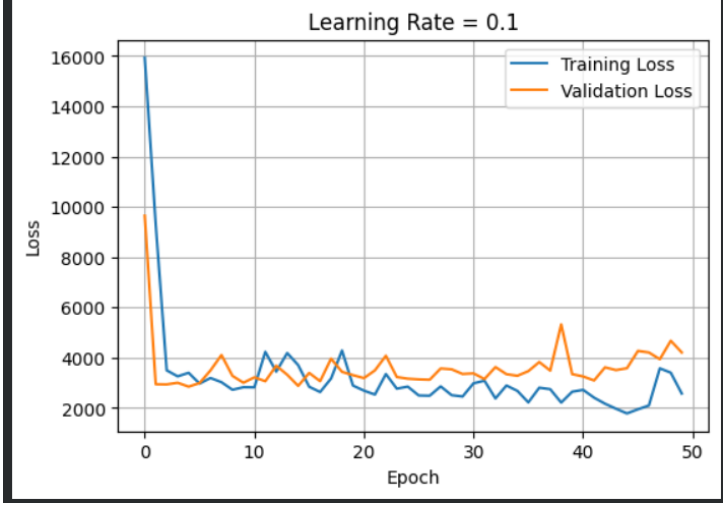
    print("MSE :", mse)
    print("MAE :", mae)
    print("R2  :", r2)

    plt.figure(figsize=(6,4))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.title(f'Epochs = {epochs}')
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.legend()
    plt.grid(True)
    plt.show()
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred, color='blue')
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
```

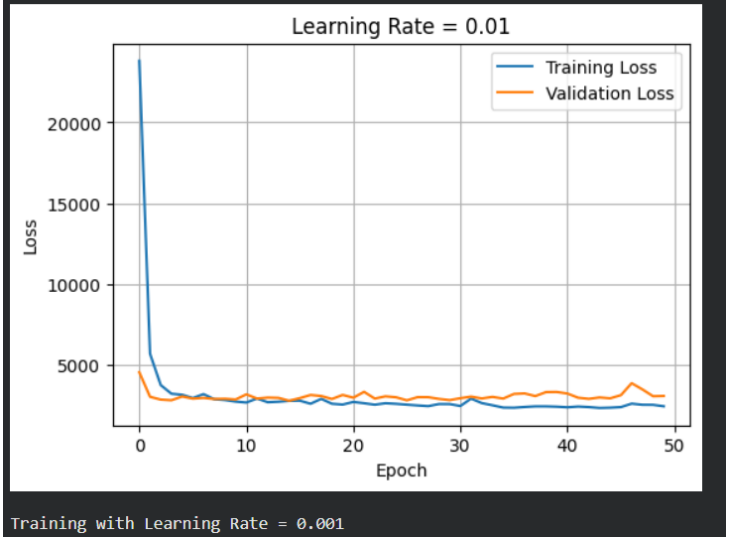
```
    color='red',
    linewidth=2
)
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.title("Actual vs Predicted")
plt.grid(True)
plt.show()
```

Output :

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
MSE : 3523.7596844062105
MAE : 46.81018957931004
R2  : 0.3349077467641791
```

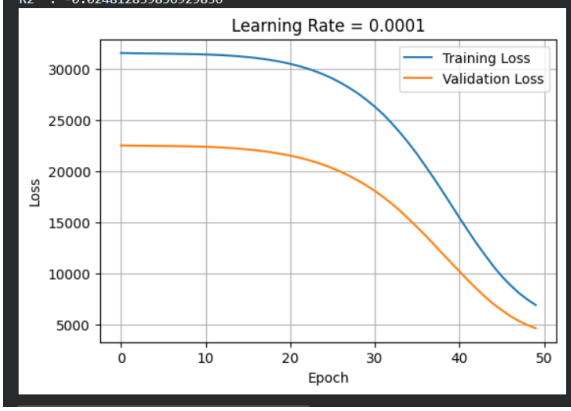



```
1/3 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/stepWARNING:tensorflow:6 out of the last 9
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
MSE : 3337.835287426079
MAE : 45.92222650935141
R2  : 0.3700000592922579
```

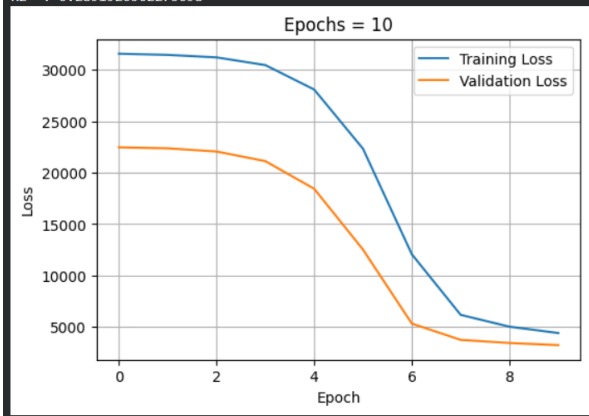



```
Training with Learning Rate = 0.001
```

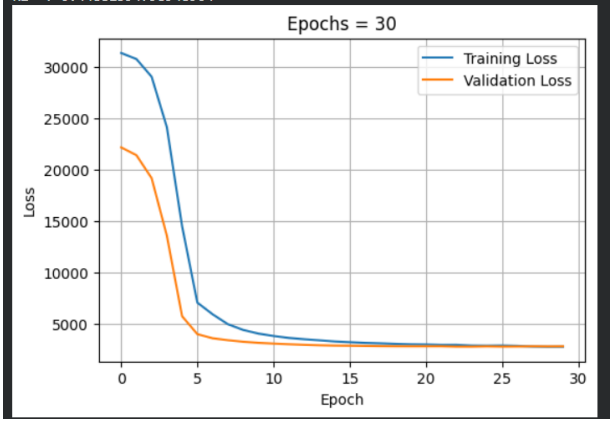
```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
MSE : 5429.613939390256
MAE : 56.96723385071486
R2  : -0.024812839856929836
```



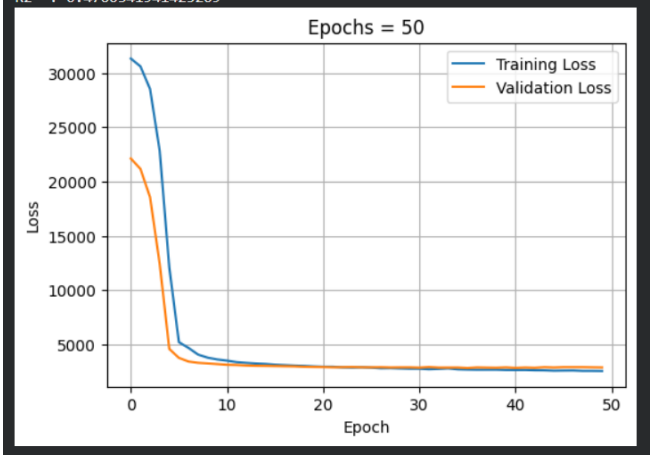

```
Training with Epochs = 10
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 52ms/step
MSE : 3765.9652030552525
MAE : 48.1422594906239
R2  : 0.2891926502275698
```




```
Training with Epochs = 30
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
MSE : 2920.267345217451
MAE : 42.61165241712934
R2  : 0.44881394798948904
```



```
Training with Epochs = 50
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step
MSE : 2807.8392609577804
MAE : 42.060685704263406
R2  : 0.4700341941425269
```




```
Training with Epochs = 100
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step
MSE : 2882.345981080673
MAE : 41.75582761443063
R2  : 0.45597141835590416
```

